

CodroidCPP SDK Manual

Version: 2.1.8 | Namespace: `Codroid`

Table of Contents

#	Section	Description
1	Quick Start	Build, connect, and run your first program
2	Core Concepts	Lifecycle, TCP model, units, exceptions
3	CodroidClient API	Complete CodroidClient API reference
4	Motion API	JointPoint, CartesianPoint, MoveInstruction, MotionWaitOptions, enums
5	Data Types and Enums	CommandResult, ClientRealtimeState, RobotFrame, exceptions
6	CRI Real-Time Data and Control	CriRealtimeDispatcher, TrajectoryGenerator, TrajectoryRequest
7	IO and Register	DI/DO/AI/AO operations, register read/write
8	Utilities	Publish/Subscribe, GlobalVariables, Kinematics, ConsoleUtf8

Environment Requirements

Platform	Compiler	Build Tool
Linux	GCC 9+ / Clang 10+	CMake 3.14+
Windows	MSVC 2019/2022/2026	CMake 3.14+
Windows	MinGW-w64 (MSYS2)	CMake 3.14+

Dependencies

- **Asio** (standalone, not Boost) — Networking
- **nlohmann/json** — JSON serialization
- **GoogleTest** (optional) — Unit testing

Build

Linux

```
chmod +x build_linux.sh
./build_linux.sh
```

Output in `build_linux/` (contains `libCodroid.so` and example programs).

Windows (MSVC)

```
build_msvc.bat
```

Script supports:

- `1`: Visual Studio 2019
- `2`: Visual Studio 2022
- `3`: Visual Studio 2026

Output in `build_msvc/Debug` and `build_msvc/Release`.

Windows (MinGW)

```
build_mingw.bat
```

Output in `build_mingw/`.

API Naming Convention

All public methods use **PascalCase** naming, consistent with C# and Python SDKs.

```
// Correct
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
```

Unit Convention

Layer	Linear	Angular
SDK public API	mm	deg (degrees)
TCP JSON protocol	mm	deg
CRI UDP binary (wire)	m	rad (radians)
<code>ClientRealtimeState</code> (parsed)	mm	deg

`CriRealtimeDispatcher` with `convert_to_si=true` (default) automatically converts mm/deg to m/rad.

Firmware Requirement

All SDK interfaces require controller firmware **>= 2.3.3.43**.

See `Codroid::MinControllerFirmware` in `CodroidDefine.h`.

Quick Start

Build SDK

Linux

```
# Clone repository
git clone <repository-url>
cd CodroidCPP

# Build
chmod +x build_linux.sh
./build_linux.sh
```

Output in `build_linux/`:

- `libCodroid.so` — Dynamic library
- `examples/` — Example programs

Windows (MSVC)

```
REM After cloning
cd CodroidCPP

REM Build
build_msvc.bat
```

Select Visual Studio version:

- `1`: Visual Studio 2019
- `2`: Visual Studio 2022
- `3`: Visual Studio 2026

Output in `build_msvc/Debug` and `build_msvc/Release`.

Windows (MinGW)

```
cd CodroidCPP
build_mingw.bat
```

Output in `build_mingw/`.

Minimal Example

Connect to the controller, read a digital input, write a digital output, and disconnect.

```
#include "Codroid/client.hpp"
```

```

#include <iostream>

int main() {
    Codroid::CodroidClient robot;

    // Connect, enter remote mode, and power on
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "Connection failed" << std::endl;
        return 1;
    }

    // Read DI port 0
    int di0 = robot.GetDi(0);
    std::cout << "DI 0 = " << di0 << std::endl;

    // Write DI value to DO port 10
    robot.SetDo(10, di0);

    // Disconnect
    robot.Disconnect();
    return 0;
}

```

Complete Workflow Example

```

#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"
#include <iostream>

int main() {
    // Windows console UTF-8 support
#ifdef _WIN32
    Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif

    Codroid::CodroidClient robot;

    // 1. Connect
    if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
        std::cerr << "Connection failed" << std::endl;
        return 1;
    }

    // 2. IO operations
    int di0 = robot.GetDi(0);
    robot.SetDo(10, di0);

    // 3. Register
    double regValue = robot.GetRegisterValue(49100);
    robot.SetRegisterValue(49100, regValue + 1);
}

```

```

// 4. Motion
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);

// 5. Blocking motion
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0},
    state.joint_position
);
robot.MovLSync(target, 150, 500);

// 6. Disconnect
robot.Disconnect();
return 0;
}

```

Run Example Programs

```

# Linux
cd build_linux
./examples/01_basic_usage 192.168.8.136

# Windows (MSVC)
cd build_msvc\Release
01_basic_usage.exe 192.168.8.136

```

Error Handling

TCP command behavior depends on `SetThrowOnCommandError` setting:

Default Mode (No Exception)

```

Codroid::CodroidClient robot;
robot.Connect("192.168.8.136");

auto result = robot.SetDo(999, 1); // Invalid port
if (!result.Ok()) {
    std::cerr << "Controller error: " << result.error_msg << std::endl;
}

```

Exception Mode

```
robot.SetThrowOnCommandError(true);

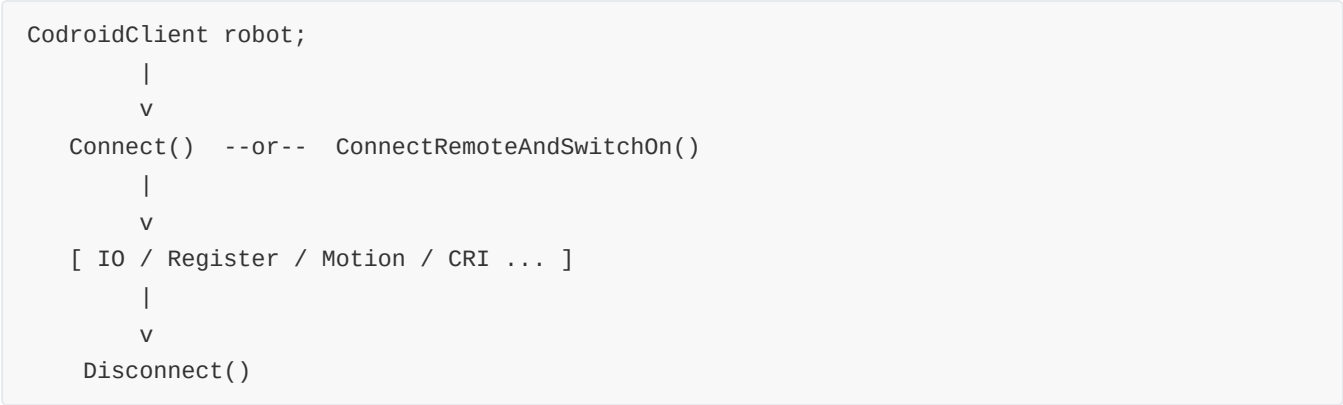
try {
    robot.SetDo(999, 1); // Invalid port
} catch (const Codroid::CodroidCommandException& ex) {
    std::cerr << "Controller error: " << ex.controller_error() << std::endl;
}
```

Exception Types

Exception	Condition
CodroidCommandException	Controller returns err field
CodroidException	General runtime error
std::runtime_error	Standard library exception

Core Concepts

CodroidClient Lifecycle



```
Codroid::CodroidClient robot;

try {
    robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
    // ... use robot ...
} catch (...) {
    robot.Disconnect(); // Always call
    throw;
}
robot.Disconnect();
```

Constructor

```
Codroid::CodroidClient robot;
```

- Default construction, no connection
- Call `Connect` or `ConnectRemoteAndSwitchOn` to establish connection
- TCP port defaults to **9001**

Properties

Method	Return Type	Description
<code>GetRobotRealtimeState()</code>	<code>ClientRealtimeState</code>	Thread-safe CRI data snapshot
<code>GetCriUdpListenPort()</code>	<code>int</code>	CRI UDP listen port

Callback

```
robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    std::cout << "Joints: ";
    for (auto j : data.joint_position) std::cout << j << " ";
    std::cout << std::endl;
});
```

Fires after each valid CRI UDP frame is parsed. Callback executes on internal receive thread — avoid long blocking.

TCP Command Model

Every SDK method that communicates with the controller follows this pattern:

1. SDK assigns a unique `id`
2. SDK serializes `{ id, ty, db }` as JSON and sends over TCP
3. Controller responds with `{ id, ty, db, err }`
4. SDK matches response by `id`
5. If `err` is non-empty → `CommandResult::error_msg` is non-empty (or throws exception)
6. If no response within 10s → timeout

CommandResult

```
struct CommandResult {
    int id = 0;           // Request id
    std::string ty;       // Response type
    std::string error_msg; // Error message (empty = success)
    std::string raw_json;  // Complete response JSON

    bool Ok() const noexcept; // Returns true if error_msg is empty
};
```

Most methods return `CommandResult`. Check `Ok()` for success.

Unit Convention

SDK public APIs use **millimeters** and **degrees**. This matches the TCP JSON protocol.

Context	Linear	Angular
SDK API, TCP JSON	mm	deg
CRI UDP wire format	m	rad
<code>ClientRealtimeState</code> (parsed)	mm	deg

Important: CRI UDP binary payloads use meters and radians. SDK automatically converts to mm/deg internally. Do not assume raw UDP floats are mm/deg.

Naming Convention

All public methods use **PascalCase**, consistent with C# and Python SDKs.

```
robot.ConnectRemoteAndSwitchOn("192.168.8.136");
int di = robot.GetDi(0);
robot.MovJ(joints, 40, 100);
robot.SetDo(10, 1);
```

Thread Safety

- `GetRobotRealtimeState()` — Thread-safe (returns clone)
- `SetCriDataReceived(callback)` — Thread-safe
- All TCP methods — Safe to call from any thread, but do not call concurrently on the same `CodroidClient`
- `CriRealtimeDispatcher::SendCommand` / `SendTrajectory` — Not thread-safe

Exception Types

Exception	Condition	Source
<code>CodroidCommandException</code>	Controller returns <code>err</code> field	TCP response
<code>CodroidException</code>	General runtime error	SDK internal
<code>std::runtime_error</code>	Standard library exception	Standard library

CodroidCommandException Properties

```
class CodroidCommandException : public CodroidException {
public:
    int request_id() const noexcept;           // Protocol request ID
    const std::string& command_ty() const noexcept; // e.g. "Robot/move"
    const std::string& controller_error() const noexcept; // Controller err field
    const std::string& raw_response_json() const noexcept; // Complete response
};
```

CodroidClient API Reference

Class: `CodroidClient`

Namespace: `Codroid`

Header: `Codroid/client.hpp`

Construction and Destruction

CodroidClient()

Default constructor, no connection.

```
Codroid::CodroidClient robot;
```

~CodroidClient()

Destructor. Automatically cleans up resources if `Disconnect()` was not called.

Connection Management

Connect

```
bool Connect(const std::string& ip, int port = 9001);
```

Establish TCP connection only, no mode switch, no power on.

Parameter	Type	Default	Description
<code>ip</code>	<code>string</code>	—	Controller IP address
<code>port</code>	<code>int</code>	9001	TCP port

Returns: `bool` — `true` on success

```
if (!robot.Connect("192.168.8.136")) {  
    std::cerr << "Connection failed" << std::endl;  
}
```

ConnectRemoteAndSwitchOn

```
bool ConnectRemoteAndSwitchOn(const std::string& ip, int port = 9001, std::string local_ip = {});
```

Connect and perform remote mode switch + power on. Recommended one-click initialization method. When `local_ip` is non-empty, it's used for `StartCriDataPush` UDP binding.

Parameter	Type	Default	Description
<code>ip</code>	<code>string</code>	—	Controller IP address
<code>port</code>	<code>int</code>	9001	TCP port
<code>local_ip</code>	<code>string</code>	empty	Local IP for CRI UDP binding

Returns: `bool` — `true` on success

```
if (!robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150")) {
    std::cerr << "Connection failed" << std::endl;
    return 1;
}
```

Disconnect

```
void Disconnect();
```

Disconnect TCP, stop CRI threads and cache. Always call before program exit.

Returns: None

```
try {
    robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
    // ... operations ...
} catch (...) {
    robot.Disconnect();
    throw;
}
robot.Disconnect();
```

NextRequestId

```
int NextRequestId();
```

Generate next monotonically increasing request ID. Avoid duplicates in multi-threaded commands.

Returns: `int` — Next available request ID

Mode Control

SwitchOn / SwitchOff

```
CommandResult SwitchOn(int id = 1);  
CommandResult SwitchOff(int id = 1);
```

Robot power on / off.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.SwitchOn();  
// ... operations ...  
robot.SwitchOff();
```

ToManual / ToAuto / ToRemote

```
CommandResult ToManual(int id = 1);  
CommandResult ToAuto(int id = 1);  
CommandResult ToRemote(int id = 1);
```

Switch to manual/auto/remote mode.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.ToRemote();
```

EnterManualModeViaAuto / EnterRemoteModeViaAuto

```
CommandResult EnterManualModeViaAuto(int id = 1);  
CommandResult EnterRemoteModeViaAuto(int id = 1);
```

Switch to auto first, then to manual/remote. Satisfies the controller's "must go through auto mode" requirement.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.EnterRemoteModeViaAuto();
```

ToSimulation / ToActual

```
CommandResult ToSimulation(int id = 1);
CommandResult ToActual(int id = 1);
```

Enter simulation/actual mode.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

StartDrag / StopDrag

```
CommandResult StartDrag(int id = 1);
CommandResult StopDrag(int id = 1);
```

Enter/exit drag teaching mode.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

ClearSystemError

```
CommandResult ClearSystemError(int id = 1);
```

Clear system error state.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

IO Operations

GetDi / GetDo / GetAi / GetAo

```
int GetDi(int port, int id = 1);
int GetDo(int port, int id = 1);
double GetAi(int port, int id = 1);
double GetAo(int port, int id = 1);
```

Read digital/analog input/output. `GetDi / GetDo` return `0` or `1`; `GetAi / GetAo` return a floating-point value.

Parameter	Type	Default	Description
<code>port</code>	<code>int</code>	—	IO port number
<code>id</code>	<code>int</code>	1	Request ID

Returns: `GetDi / GetDo` → `int` (0 or 1); `GetAi / GetAo` → `double`

```
int di0 = robot.GetDi(0);
std::cout << "DI 0 = " << di0 << std::endl;

double ai1 = robot.GetAi(1);
std::cout << "AI 1 = " << ai1 << std::endl;
```

SetDo / SetAo

```
CommandResult SetDo(int port, int value, int id = 1);
CommandResult SetAo(int port, double value, int id = 1);
```

Set digital/analog output. `SetDo` `value` must be `0` or `1`.

Parameter	Type	Default	Description
<code>port</code>	<code>int</code>	—	IO port number
<code>value</code>	<code>int</code> / <code>double</code>	—	Write value (<code>SetDo</code> : 0 or 1, <code>SetAo</code> : float)
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.SetDo(10, 1);    // Set ON
robot.SetDo(10, 0);    // Set OFF
robot.SetAo(0, 3.14);
```

Register Operations

GetRegisterValue

```
double GetRegisterValue(int address, int id = 1);
```

Read a single register value.

Parameter	Type	Default	Description
address	int	—	Register address
id	int	1	Request ID

Returns: double — Register value

```
double value = robot.GetRegisterValue(49100);
std::cout << "Register 49100 = " << value << std::endl;
```

GetRegisterValues

```
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id = 1);
```

Batch read register values.

Parameter	Type	Default	Description
addresses	vector<int>	—	List of register addresses
id	int	1	Request ID

Returns: vector<ClientRegisterInfo> — Register values in the same order as input addresses

```
std::vector<int> addresses = {49100, 49101, 49102};
auto values = robot.GetRegisterValues(addresses);
for (auto& v : values)
    std::cout << "Address " << v.address << " = " << v.value << std::endl;
```

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

Set register value.

Parameter	Type	Default	Description
<code>address</code>	<code>int</code>	—	Register address
<code>value</code>	<code>double</code>	—	Write value
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.SetRegisterValue(49100, 42);
robot.SetRegisterValue(49101, 3.14);
```

Motion Control (Non-Blocking)

All motion methods return immediately after sending the command. Use `*Sync` variants for blocking wait.

MovJ — Joint Motion

```
CommandResult MovJ(const ClientJointPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);

CommandResult MovJ(const ClientCartesianPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);
```

Parameter	Type	Default	Description
<code>target</code>	<code>ClientJointPoint</code> / <code>ClientCartesianPoint</code>	—	Target position
<code>speed</code>	<code>double</code>	—	Speed (deg/s)
<code>acceleration</code>	<code>double</code>	—	Acceleration
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted

Parameter	Type	Default	Description
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
// Joint target
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);

// Cartesian target (joint motion to TCP pose)
auto pose = Codroid::CartesianPoint::MmDeg({400, 0, 300, 180, 0, 0});
robot.MovJ(pose, 40, 100);
```

MovL — Linear Motion

```
CommandResult MovL(const ClientCartesianPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);

CommandResult MovL(const ClientJointPoint& target, double speed, double acceleration,
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool =
                  {}, int id = 1);
```

Parameter	Type	Default	Description
<code>target</code>	<code>ClientCartesianPoint</code> / <code>ClientJointPoint</code>	—	Target position
<code>speed</code>	<code>double</code>	—	Speed (mm/s)
<code>acceleration</code>	<code>double</code>	—	Acceleration
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted

Parameter	Type	Default	Description
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0}, state.joint_position);
robot.MovL(target, 150, 500);
```

MovC — Circular Motion

```
CommandResult MovC(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,
    double speed, double acceleration,
    double blend = -1, double relativeBlend = -1,
    const std::vector<double>& coor = {}, const std::vector<double>& tool =
    {}, int id = 1);
```

Parameter	Type	Default	Description
<code>middle</code>	<code>ClientCartesianPoint</code>	—	Intermediate point (on the arc)
<code>target</code>	<code>ClientCartesianPoint</code>	—	End point
<code>speed</code>	<code>double</code>	—	Speed (mm/s)
<code>acceleration</code>	<code>double</code>	—	Acceleration
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
auto mid = Codroid::CartesianPoint::MmDeg({450, 100, 300, 180, 0, 0});
auto end = Codroid::CartesianPoint::MmDeg({500, 0, 300, 180, 0, 0});
robot.MovC(mid, end, 100, 300);
```

MovCircle — Full Circle Motion

```
CommandResult MovCircle(const ClientCartesianPoint& middle, const ClientCartesianPoint&
target,
                        int circle_num, double speed, double acceleration,
                        double blend = -1, double relativeBlend = -1,
                        const std::vector<double>& coor = {}, const std::vector<double>&
tool = {}, int id = 1);
```

Parameter	Type	Default	Description
middle	ClientCartesianPoint	—	Intermediate point
target	ClientCartesianPoint	—	End point
circle_num	int	—	Number of full circles
speed	double	—	Speed (mm/s)
acceleration	double	—	Acceleration
blend	double	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
relativeBlend	double	-1	Relative blend ratio (0-100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
coor	vector<double>	{}	User coordinate frame. Empty means the field is omitted
tool	vector<double>	{}	Tool coordinate frame. Empty means the field is omitted
id	int	1	Request ID

Returns: `CommandResult` — Controller response

```
auto mid = Codroid::CartesianPoint::MmDeg({450, 100, 300, 180, 0, 0});
auto end = Codroid::CartesianPoint::MmDeg({500, 0, 300, 180, 0, 0});
robot.MovCircle(mid, end, 1, 80, 200);
```

Move / MovePath — Multi-Segment Path

```
CommandResult Move(const std::vector<ClientMoveInstruction>& path, int id = 1);
CommandResult MovePath(const std::vector<ClientMoveInstruction>& path, int id = 1);
```

Send a set of motion instructions as a single path command. `MovePath` is an alias for `Move`.

Parameter	Type	Default	Description
<code>path</code>	<code>vector<ClientMoveInstruction></code>	—	List of motion instructions
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
auto state = robot.GetRobotRealtimeState();
std::vector<Codroid::ClientMoveInstruction> path = {
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(
        Codroid::CartesianPoint::MmDegWithRef({400, 0, 300, 180, 0, 0},
        state.joint_position),
        150, 500)
};
robot.Move(path);
```

Blocking Motion (Sync)

*Sync methods send the motion command and **block until CRI confirms the robot has stopped**. Returns `true` on success, throws on error/timeout.

Prerequisite: Call `StartCriDataPush` and wait for the first frame (`WaitForCriData`).

v2.1.8 behavior change: Completion is determined solely by the CRI `InMotion` flag (was moving + consecutive `SettledSamples` stable reads). Joint/TCP position vs target is **no longer checked**.

`MotionWaitOptions` tolerance fields (`joint_tolerance_deg`, `cartesian_position_tolerance_mm`, `cartesian_orientation_tolerance_deg`) are `[[deprecated]]` and have no effect.

MoveSync

```
bool MoveSync(const std::vector<ClientMoveInstruction>& path, const MotionWaitOptions& wait = {});
```

Send a multi-segment path and block until the last segment's target is reached.

Parameter	Type	Default	Description
<code>path</code>	<code>vector<ClientMoveInstruction></code>	—	List of motion instructions

Parameter	Type	Default	Description
<code>wait</code>	<code>MotionWaitOptions</code>	default	Wait options (timeout, tolerance, etc.)

Returns: `bool` — `true` on successful arrival at target

Exception behavior: Timeout controlled by `MotionWaitOptions.timeout_s`; returns `false` on abnormal robot state (collision, emergency stop, alarm)

```
robot.MoveSync({
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(target, 150, 500)
});
```

MovJSync

```
bool MovJSync(const ClientJointPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});

bool MovJSync(const ClientCartesianPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

Parameter	Type	Default	Description
<code>target</code>	<code>ClientJointPoint</code> / <code>ClientCartesianPoint</code>	—	Target position
<code>speed</code>	<code>double</code>	—	Speed (deg/s)
<code>acc</code>	<code>double</code>	—	Acceleration
<code>wait</code>	<code>MotionWaitOptions</code>	default	Wait options (timeout, tolerance, etc.)
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted

Parameter	Type	Default	Description
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted

Returns: `bool` — `true` on successful arrival at target

Exception behavior: Returns `false` on abnormal robot state (collision, emergency stop, alarm)

```
Codroid::MotionWaitOptions wait;
wait.timeout_s = 90.0;
wait.joint_tolerance_deg = 0.3;

robot.MovJSync(Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100, wait);
```

MovLSync

```
bool MovLSync(const ClientCartesianPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});

bool MovLSync(const ClientJointPoint& target, double speed, double acc,
              const MotionWaitOptions& wait = {},
              double blend = -1, double relativeBlend = -1,
              const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

Parameter	Type	Default	Description
<code>target</code>	<code>ClientCartesianPoint</code> / <code>ClientJointPoint</code>	—	Target position
<code>speed</code>	<code>double</code>	—	Speed (mm/s)
<code>acc</code>	<code>double</code>	—	Acceleration
<code>wait</code>	<code>MotionWaitOptions</code>	default	Wait options
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted

Parameter	Type	Default	Description
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted

Returns: `bool` — `true` on successful arrival at target

Exception behavior: Returns `false` on abnormal robot state (collision, emergency stop, alarm)

```

auto state = robot.GetRobotRealtimeState();
auto target = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0}, state.joint_position);

Codroid::MotionWaitOptions wait;
wait.timeout_s = 60.0;
wait.cartesian_position_tolerance_mm = 2.0;
wait.cartesian_orientation_tolerance_deg = 1.5;

robot.MovLSync(target, 150, 500, wait);

```

MovCSync

```

bool MovCSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,
    double speed, double acc, const MotionWaitOptions& wait = {},
    double blend = -1, double relativeBlend = -1,
    const std::vector<double>& coor = {}, const std::vector<double>& tool = {});

```

Parameter	Type	Default	Description
<code>middle</code>	<code>ClientCartesianPoint</code>	—	Intermediate point (on the arc)
<code>target</code>	<code>ClientCartesianPoint</code>	—	End point
<code>speed</code>	<code>double</code>	—	Speed (mm/s)
<code>acc</code>	<code>double</code>	—	Acceleration
<code>wait</code>	<code>MotionWaitOptions</code>	default	Wait options
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted

Parameter	Type	Default	Description
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted

Returns: `bool` — `true` on successful arrival at target

Exception behavior: Returns `false` on abnormal robot state (collision, emergency stop, alarm)

```
auto mid = Codroid::CartesianPoint::MmDeg({450, 100, 300, 180, 0, 0});
auto end = Codroid::CartesianPoint::MmDeg({500, 0, 300, 180, 0, 0});
robot.MovCSync(mid, end, 100, 300);
```

MovCircleSync

```
bool MovCircleSync(const ClientCartesianPoint& middle, const ClientCartesianPoint& target,
                  int circle_num, double speed, double acc, const MotionWaitOptions& wait = {},
                  double blend = -1, double relativeBlend = -1,
                  const std::vector<double>& coor = {}, const std::vector<double>& tool = {});
```

Parameter	Type	Default	Description
<code>middle</code>	<code>ClientCartesianPoint</code>	—	Intermediate point
<code>target</code>	<code>ClientCartesianPoint</code>	—	End point
<code>circle_num</code>	<code>int</code>	—	Number of full circles
<code>speed</code>	<code>double</code>	—	Speed (mm/s)
<code>acc</code>	<code>double</code>	—	Acceleration
<code>wait</code>	<code>MotionWaitOptions</code>	default	Wait options
<code>blend</code>	<code>double</code>	-1	Blend radius. Mutually exclusive with <code>relativeBlend</code> — if both are set, <code>relativeBlend</code> is ignored. -1 means no transition
<code>relativeBlend</code>	<code>double</code>	-1	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this parameter is ignored
<code>coor</code>	<code>vector<double></code>	<code>{}</code>	User coordinate frame. Empty means the field is omitted
<code>tool</code>	<code>vector<double></code>	<code>{}</code>	Tool coordinate frame. Empty means the field is omitted

Returns: `bool` — `true` on successful arrival at target

Exception behavior: Returns `false` on abnormal robot state (collision, emergency stop, alarm)

Motion Control

PauseRobotMotion

```
CommandResult PauseRobotMotion(int id = 1);
```

Pause current motion.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

ResumeRobotMotion

```
CommandResult ResumeRobotMotion(int id = 1);
```

Resume paused motion.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

StopRobotMove

```
CommandResult StopRobotMove(int id = 1);
```

Immediately stop current motion.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

MoveTo (Planned Motion)

MoveTo

```
CommandResult MoveTo(const MoveToParams& params, int id = 1);
```

Move to a preset or planned position. Heartbeat required when using `MoveToType::Joint` or `Line`.

Parameter	Type	Default	Description
<code>params</code>	<code>MoveToParams</code>	—	Target type and optional target point
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
// Move to home
robot.MoveTo(Codroid::MoveToParams{Codroid::MoveToType::Home});

// Move to safe position
robot.MoveTo(Codroid::MoveToParams{Codroid::MoveToType::Safe});
```

MoveToHeartbeat

```
CommandResult MoveToHeartbeat(int id = 1);
```

Send heartbeat to maintain MoveTo motion. Call every ≥ 500 ms when using `MoveToType::Joint` or `Line`.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

StopMoveTo

```
CommandResult StopMoveTo(int id = 1);
```

Stop current MoveTo motion.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

Jog

Jog

```
CommandResult Jog(const JogParams& params, int id = 1);
```

Start jog. Requires heartbeat every ~500ms.

Parameter	Type	Default	Description
params	JogParams	—	Jog parameters (mode, speed, axis index, coordinate frame)
id	int	1	Request ID

Returns: CommandResult — Controller response

```
Codroid::JogParams jog;
jog.mode = Codroid::JogMode::Joint;
jog.speed = 10.0;
jog.index = 0;
jog.coorType = Codroid::CoorType::User;
jog.coorId = 0;
robot.Jog(jog);

// Keep sending heartbeat
while (jogging) {
    std::this_thread::sleep_for(std::chrono::milliseconds(500));
    robot.JogHeartbeat();
}
robot.StopJog();
```

StopJog

```
CommandResult StopJog(int id = 1);
```

Stop jog.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: CommandResult — Controller response

JogHeartbeat

```
CommandResult JogHeartbeat(int id = 1);
```

Send heartbeat to maintain jog state. Call every ~500ms.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

CRI Real-Time Data

StartCriDataPush

```
CommandResult StartCriDataPush(const std::string& udpIp, int udpPort, int id = 1);
```

Start local UDP listening and request controller to push CRI real-time data.

Parameter	Type	Default	Description
udpIp	string	—	Local IP address for receiving UDP data
udpPort	int	—	Local port number
id	int	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.StartCriDataPush("192.168.8.150", 18888);
robot.WaitForCriData(5.0); // wait for first frame

robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    std::cout << "Joints: ";
    for (auto j : data.joint_position) std::cout << j << " ";
    std::cout << std::endl;
});
```

StopCriDataPush

```
CommandResult StopCriDataPush(int id = 1);
CommandResult StopCriDataPush(const std::string& udpIp, int udpPort, int id = 1);
```

Request controller to stop CRI data push and close local UDP listening.

Parameter	Type	Default	Description
<code>udpIp</code>	<code>string</code>	—	Local IP address (optional overload)
<code>udpPort</code>	<code>int</code>	—	Local port number (optional overload)
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

WaitForCriData

```
void WaitForCriData(double timeout_s = 5.0);
```

Block until the first CRI data frame arrives.

Parameter	Type	Default	Description
<code>timeout_s</code>	<code>double</code>	5.0	Maximum wait time in seconds

Returns: None

CRI Real-Time Control

StartCriControl

```
CommandResult StartCriControl(int filterType, int durationMs, int startBuffer, int id = 1);
```

Enable CRI real-time control mode.

Parameter	Type	Default	Description
<code>filterType</code>	<code>int</code>	—	0=off, 1=average, 2=2nd-order low-pass, 3=elliptic
<code>durationMs</code>	<code>int</code>	—	Control period (1~16ms, must divide 1000)
<code>startBuffer</code>	<code>int</code>	—	Starting buffer frames (1~100)
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.StartCriControl(1, 4, 5);
```

StopCriControl

```
CommandResult StopCriControl(int id = 1);
```

Disable CRI real-time control mode.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `CommandResult` — Controller response

GetRobotRealtimeState

```
ClientRealtimeState GetRobotRealtimeState() const;
```

Get thread-safe CRI data snapshot.

Returns: `ClientRealtimeState` — CRI real-time data clone

```
auto state = robot.GetRobotRealtimeState();
std::cout << "J1 = " << state.joint_position[0] << " deg" << std::endl;
std::cout << "TCP X = " << state.tcp_pose[0] << " mm" << std::endl;
```

SetCriDataReceived

```
void SetCriDataReceived(std::function<void(const ClientRealtimeState&)> cb);
```

Set CRI data callback. Fires after each valid CRI UDP frame is parsed.

Parameter	Type	Default	Description
cb	<code>function<void(const ClientRealtimeState&)></code>	—	Callback function; executes on internal receive thread, avoid long blocking

Returns: None

```
robot.SetCriDataReceived([](const Codroid::ClientRealtimeState& data) {
    if (data.has_alarm) {
        std::cerr << "Robot has alarm!" << std::endl;
    }
});
```

GetCriUdpListenPort

```
int GetCriUdpListenPort() const;
```

Get the local UDP listen port for CRI push binding. May be 0 if push is not started.

Returns: `int` — UDP listen port

Project/Script

EnterRemoteScriptMode

```
CommandResult EnterRemoteScriptMode(int id = 1);
```

Request to enter remote script mode.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

RunScript

```
CommandResult RunScript(const std::string& mainScript,
                        const std::unordered_map<std::string, std::string>& subThreads = {},
                        const std::unordered_map<std::string, std::string>& subPrograms = {},
                        const std::unordered_map<std::string, std::string>& interrupts = {},
                        const nlohmann::json& vars = {},
                        int id = 1);
```

Send script for immediate execution.

Parameter	Type	Default	Description
<code>mainScript</code>	<code>string</code>	—	Main script content
<code>subThreads</code>	<code>unordered_map<string, string></code>	<code>{}</code>	Sub-thread scripts
<code>subPrograms</code>	<code>unordered_map<string, string></code>	<code>{}</code>	Sub-program scripts
<code>interrupts</code>	<code>unordered_map<string, string></code>	<code>{}</code>	Interrupt handler scripts
<code>vars</code>	<code>nlohmann::json</code>	<code>{}</code>	Injection variables
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.RunScript("movej(j1, v50) sub1() end");
```

Run / RunByIndex / RunStep

```
CommandResult Run(const std::string& projectId, int id = 1);  
CommandResult RunByIndex(int index, int id = 1);  
CommandResult RunStep(const std::string& projectId, int id = 1);
```

Start project by ID/index or step-execute.

Run / RunStep parameters:

Parameter	Type	Default	Description
<code>projectId</code>	<code>string</code>	—	Project ID
<code>id</code>	<code>int</code>	1	Request ID

RunByIndex parameters:

Parameter	Type	Default	Description
<code>index</code>	<code>int</code>	—	Project index
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.Run("project_001");  
robot.RunByIndex(0);  
robot.RunStep("project_001");
```

PauseProject / ResumeProject / StopProject

```
CommandResult PauseProject(int id = 1);  
CommandResult ResumeProject(int id = 1);  
CommandResult StopProject(int id = 1);
```

Pause, resume, stop project.

Parameter	Type	Default	Description
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

Publish/Subscribe

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(  
    std::string topicTy,  
    std::function<void(const ClientPublishNotification&)> handler,  
    int tc_milliseconds = 100);
```

Subscribe to TCP topic push. Returns a disposable subscription handle.

Parameter	Type	Default	Description
topicTy	string	—	Topic name, e.g. <code>PublishTopics::RobotStatus</code>
handler	function<void(const ClientPublishNotification&)>	—	Callback for notifications; should not block for long
tcMilliseconds	int	100	Push period (ms)

Returns: `ClientPublishSubscription` — Disposable subscription handle

```
auto sub = robot.SubscribePublishTopic(  
    Codroid::PublishTopics::RobotStatus,  
    [](const Codroid::ClientPublishNotification& n) {  
        std::cout << "Status: " << n.db_json << std::endl;  
    });  
  
// Subscription is active until sub.Dispose() or destruction  
std::this_thread::sleep_for(std::chrono::seconds(10));  
sub.Dispose();
```

Global Variables

GetGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);
```

Get all global variables.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `nlohmann::json` — Global variables JSON

```
auto vars = robot.GetGlobalVars();
std::cout << vars.dump(2) << std::endl;
```

SaveGlobalVars

```
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);
```

Save global variables.

Parameter	Type	Default	Description
vars	map<string, Variable>	—	Key-value pairs to save
id	int	1	Request ID

Returns: CommandResult — Controller response

```
robot.SaveGlobalVars({
    {"counter", Codroid::Variable(42, "Counter")},
    {"name", Codroid::Variable(std::string("test"), "Name")}
});
```

RemoveGlobalVars

```
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

Delete specified global variables. Deleting non-existent names does not cause an error.

Parameter	Type	Default	Description
names	vector<string>	—	Variable names to delete
id	int	1	Request ID

Returns: CommandResult — Controller response

```
robot.RemoveGlobalVars({"counter", "name"});
```

Kinematics

ForwardKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);
```

Forward kinematics: joint space -> Cartesian space. Returns `[x,y,z,rx,ry,rz]` in mm + deg.

Parameter	Type	Default	Description
<code>params</code>	<code>FKParams</code>	—	Joint angles, coordinate frame, tool parameters
<code>id</code>	<code>int</code>	1	Request ID

Returns: `vector<double>` — Cartesian pose `[x,y,z,rx,ry,rz]`

```
Codroid::FKParams fk({0, 0, 90, 0, 90, 0});  
auto tcp = robot.ForwardKinematics(fk);
```

InverseKinematics

```
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

Inverse kinematics: Cartesian -> joint space. Returns 6 joint angles in degrees.

Parameter	Type	Default	Description
<code>params</code>	<code>IKParams</code>	—	TCP pose, reference joints, external axis parameters
<code>id</code>	<code>int</code>	1	Request ID

Returns: `vector<double>` — 6 joint angles in degrees

```
Codroid::IKParams ik({400, 0, 300, 180, 0, 0});  
ik.rj = {0, 0, 90, 0, 90, 0};  
auto joints = robot.InverseKinematics(ik);
```

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

Calculate relative pose/offset in user or tool coordinate frame.

Parameter	Type	Default	Description
params	RelativePoseParams	—	Current pose, offset, coordinate frame type parameters
id	int	1	Request ID

Returns: `vector<double>` — Calculated pose `[x, y, z, rx, ry, rz]`

```
Codroid::RelativePoseParams rp;
rp.pos = {400, 0, 300, 180, 0, 0};
rp.offset = {50, 0, 0, 0, 0, 0};
rp.coorType = Codroid::CoorType::User;
auto newPose = robot.CalculateRelativePose(rp);
```

Robot Settings

GetRobotParameters

```
ClientRobotParameters GetRobotParameters(int id = 1);
```

Get all settings parameters. Returns tool, payload, user coordinate frames and default IDs.

Parameter	Type	Default	Description
id	int	1	Request ID

Returns: `ClientRobotParameters` — Full robot parameter set (`valid` is `false` on failure)

```
auto param = robot.GetRobotParameters();
if (param.valid) {
    std::cout << "Default tool: " << param.default_tool_id << std::endl;
    std::cout << "Max payload: " << param.max_payload << " kg" << std::endl;
}
```

SetDefaultPayloadId / SetDefaultToolId / SetDefaultUserCoordinateId

```
CommandResult SetDefaultPayloadId(int payloadId, int id = 1);
CommandResult SetDefaultToolId(int toolId, int id = 1);
CommandResult SetDefaultUserCoordinateId(int coordinateId, int id = 1);
```

Set default payload/tool/user coordinate frame ID (1~15).

Parameter	Type	Default	Description
<code>payloadId</code> / <code>toolId</code> / <code>coordinateId</code>	<code>int</code>	—	Slot number, range 1~15
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.SetDefaultToolId(2);
robot.SetDefaultPayloadId(1);
robot.SetDefaultUserCoordinateId(0);
```

SaveToolFrames / SetToolFrame

```
CommandResult SaveToolFrames(const std::vector<ClientRobotFrame>& frames, int id = 1);
CommandResult SetToolFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
```

Save complete tool frame table / modify a single tool frame (`frame_id` 1~15).

SaveToolFrames parameters:

Parameter	Type	Default	Description
<code>frames</code>	<code>vector<ClientRobotFrame></code>	—	Tool frame list
<code>id</code>	<code>int</code>	1	Request ID

SetToolFrame parameters:

Parameter	Type	Default	Description
<code>frame_id</code>	<code>int</code>	—	Tool frame number, range 1~15
<code>frame</code>	<code>ClientRobotFrame</code>	—	Frame definition
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

```
Codroid::CodroidClient::ClientRobotFrame tool;
tool.id = 1; tool.x = 0; tool.y = 0; tool.z = 100;
tool.a = 0; tool.b = 0; tool.c = 0;
robot.SetToolFrame(1, tool);
```

SavePayloadFrames / SetPayloadFrame

```
CommandResult SavePayloadFrames(const std::vector<ClientRobotPayload>& frames, int id = 1);
CommandResult SetPayloadFrame(int frame_id, const ClientRobotPayload& frame, int id = 1);
```

Save complete payload frame table / modify a single payload frame (`frame_id` 1~15).

SavePayloadFrames parameters:

Parameter	Type	Default	Description
<code>frames</code>	<code>vector<ClientRobotPayload></code>	—	Payload frame list
<code>id</code>	<code>int</code>	1	Request ID

SetPayloadFrame parameters:

Parameter	Type	Default	Description
<code>frame_id</code>	<code>int</code>	—	Payload frame number, range 1~15
<code>frame</code>	<code>ClientRobotPayload</code>	—	Payload frame definition
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

SetUserCoordinateFrame

```
CommandResult SetUserCoordinateFrame(int frame_id, const ClientRobotFrame& frame, int id = 1);
```

Modify a single user coordinate frame (read-then-modify; `frame_id` 1~15).

Parameter	Type	Default	Description
<code>frame_id</code>	<code>int</code>	—	User coordinate frame number, range 1~15
<code>frame</code>	<code>ClientRobotFrame</code>	—	Frame definition
<code>id</code>	<code>int</code>	1	Request ID

Returns: `CommandResult` — Controller response

SetPayload / SetManualMoveRate / SetAutoMoveRate / SetCollisionSensitivity

```
CommandResult SetPayload(int payloadId, int id = 1);
CommandResult SetManualMoveRate(int pct, int id = 1);
CommandResult SetAutoMoveRate(int pct, int id = 1);
CommandResult SetCollisionSensitivity(int sensitivity, int id = 1);
```

Runtime parameter settings.

SetPayload parameters:

Parameter	Type	Default	Description
payloadId	int	—	Payload slot number (0~15)
id	int	1	Request ID

SetManualMoveRate / SetAutoMoveRate parameters:

Parameter	Type	Default	Description
pct	int	—	Speed override percentage, range 1~100
id	int	1	Request ID

SetCollisionSensitivity parameters:

Parameter	Type	Default	Description
sensitivity	int	—	Collision detection sensitivity, range 0~100
id	int	1	Request ID

Returns: `CommandResult` — Controller response

```
robot.SetPayload(1);
robot.SetManualMoveRate(50); // 50% speed
robot.SetAutoMoveRate(100); // Full speed
robot.SetCollisionSensitivity(50);
```


Error Handling Settings

SetThrowOnCommandError / ThrowOnCommandError

```
void SetThrowOnCommandError(bool enable);  
bool ThrowOnCommandError() const noexcept;
```

Set/get error handling mode. When set to `true`, command failures throw `CodroidCommandException`.

Parameter	Type	Default	Description
<code>enable</code>	<code>bool</code>	—	<code>true</code> to enable exception mode

Returns: `SetThrowOnCommandError` returns `none`; `ThrowOnCommandError` returns `bool`

```
robot.SetThrowOnCommandError(true);  
try {  
    robot.SetDo(999, 1);  
} catch (const Codroid::CodroidCommandException& ex) {  
    std::cerr << "Controller error: " << ex.controller_error() << std::endl;  
}
```

Motion API Reference

This section covers all motion-related types in CodroidCPP SDK, including joint/Cartesian point definitions, motion instructions, jog parameters, and motion wait options.

Point Types

JointPoint

Joint space target point (six axes, unit: degrees).

```
struct JointPoint {
    std::vector<double> jp; // Six joint angles (degrees)

    static JointPoint Degrees(std::vector<double> joints_deg);
};
```

Property	Type	Description
jp	vector<double>	Six joint angles (degrees)

Factory Method	Description
JointPoint::Degrees(vector<double> joints_deg)	Create from 6 joint angles (degrees). Array must be exactly length 6.

Usage:

```
auto joints = Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0});
robot.MovJ(joints, 40, 100);
```

CartesianPoint

Cartesian end-effector target point (TCP pose: mm + degrees).

```
struct CartesianPoint {
    std::vector<double> cp; // [x, y, z, rx, ry, rz]
    std::vector<double> rj; // Inverse kinematics reference joints (degrees)

    static CartesianPoint MmDeg(std::vector<double> pose_mm_deg);
    static CartesianPoint MmDegWithRef(std::vector<double> pose_mm_deg, std::vector<double> ref_joints_deg);
};
```

Property	Type	Description
<code>cp</code>	<code>vector<double></code>	TCP pose <code>[x, y, z, rx, ry, rz]</code> — position in mm, orientation in degrees
<code>rj</code>	<code>vector<double></code>	Reference joints for IK (6 joint angles, degrees). Empty uses defaults

Factory Method	Description
<code>CartesianPoint::MmDeg(vector<double> pose)</code>	Create from TCP pose only (uses default reference joints)
<code>CartesianPoint::MmDegWithRef(vector<double> pose, vector<double> refJoints)</code>	Create with explicit reference joints

Usage:

```
// Without reference joints
auto pose1 = Codroid::CartesianPoint::MmDeg({400, 0, 300, 180, 0, 0});

// With reference joints (recommended)
auto state = robot.GetRobotRealtimeState();
auto pose2 = Codroid::CartesianPoint::MmDegWithRef(
    {400, 0, 300, 180, 0, 0},
    state.joint_position
);
```

MovePoint

Single target point in path segment (protocol layer).

```
struct MovePoint {
    std::vector<double> jp;
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;

    static MovePoint Joint(JointPoint joint);
    static MovePoint Cartesian(CartesianPoint cart);
};
```

Property	Type	Description
<code>jp</code>	<code>vector<double></code>	Joint angles (degrees), empty for Cartesian targets
<code>cp</code>	<code>vector<double></code>	TCP pose (mm + degrees), empty for joint targets
<code>rj</code>	<code>vector<double></code>	Reference joints for IK

Property	Type	Description
<code>ep</code>	<code>vector<double></code>	External axis

Path Instructions

ClientMoveInstruction

Single segment in a path.

```
struct ClientMoveInstruction {
    ClientMoveType type = ClientMoveType::MovJ;
    double speed = 60.0;
    double acceleration = 150.0;
    double blend = -1.0;
    double relative_blend = -1.0;
    int circle_num = 1;
    ClientMovePoint target;
    ClientMovePoint middle;
    std::vector<double> coor;
    std::vector<double> tool;

    static ClientMoveInstruction MovJ(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovJ(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientCartesianPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovL(ClientJointPoint target, double speed, double
acceleration, double blend = -1.0);
    static ClientMoveInstruction MovC(ClientCartesianPoint middle, ClientCartesianPoint
target, ...);
    static ClientMoveInstruction MovCircle(ClientCartesianPoint middle, ClientCartesianPoint
target, int circle_num, ...);
};
```

Property	Type	Default	Description
<code>type</code>	<code>ClientMoveType</code>	<code>MovJ</code>	Motion type
<code>speed</code>	<code>double</code>	60.0	Speed (linear mm/s, joint deg/s)
<code>acceleration</code>	<code>double</code>	150.0	Acceleration
<code>blend</code>	<code>double</code>	-1.0	Blend radius. Mutually exclusive with <code>relative_blend</code> — if both are set, <code>relative_blend</code> is ignored. -1 means no transition

Property	Type	Default	Description
<code>relative_blend</code>	<code>double</code>	-1.0	Relative blend ratio (0–100). Mutually exclusive with <code>blend</code> — if both are set, this property is ignored
<code>circle_num</code>	<code>int</code>	1	Number of full circles (only for <code>MovCircle</code>)
<code>target</code>	<code>ClientMovePoint</code>	—	Target point for this segment
<code>middle</code>	<code>ClientMovePoint</code>	—	Intermediate/via point (required for <code>MovC</code> and <code>MovCircle</code>)
<code>coor</code>	<code>vector<double></code>	{}	User coordinate frame definition
<code>tool</code>	<code>vector<double></code>	{}	Tool coordinate frame definition

Factory method parameter reference:

Parameter	Type	Default	Description
<code>target</code>	<code>ClientJointPoint</code> / <code>ClientCartesianPoint</code>	—	Target point
<code>speed</code>	<code>double</code>	—	Speed (mm/s or deg/s)
<code>acceleration</code>	<code>double</code>	—	Acceleration
<code>blend</code>	<code>double</code>	-1.0	Blend radius. -1 means no transition

Usage:

```
auto state = robot.GetRobotRealtimeState();

std::vector<Codroid::ClientMoveInstruction> path = {
    Codroid::ClientMoveInstruction::MovJ(
        Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100),
    Codroid::ClientMoveInstruction::MovL(
        Codroid::CartesianPoint::MmDegWithRef({400, 0, 300, 180, 0, 0},
state.joint_position),
        150, 500)
};
robot.Move(path);
```

Motion Type Enums

ClientMoveType

```
enum class ClientMoveType {
    MovJ,          // Joint motion
    MovL,          // Linear (Cartesian)
    MovC,          // Circular (via midpoint)
    MovCircle      // Full circle / multi-revolution
};
```

Name	Description
MovJ	Joint motion
MovL	Linear (Cartesian) motion
MovC	Circular motion (via intermediate point)
MovCircle	Full circle / multi-revolution motion

Blocking Motion Parameters

MotionWaitOptions

Configures how the SDK waits for motion completion.

v2.1.8 change: Completion is determined solely by the CRI `InMotion` flag. Tolerance fields are deprecated.

```
struct MotionWaitOptions {
    double timeout_s = 60.0;
    double poll_interval_s = 0.05;
    double cri_stale_timeout_s = 0.5;
    int settled_samples = 3;
    [[deprecated]] double joint_tolerance_deg = 0.2;           // deprecated
    [[deprecated]] double cartesian_position_tolerance_mm = 1.0; // deprecated
    [[deprecated]] double cartesian_orientation_tolerance_deg = 1.0; // deprecated
};
```

Property	Type	Default	Description
timeout_s	double	60.0	Maximum time to wait for motion completion (seconds)
poll_interval_s	double	0.05	Polling interval for motion status (seconds)

Property	Type	Default	Description
<code>cri_stale_timeout_s</code>	<code>double</code>	0.5	Maximum time before CRI data is considered stale (seconds)
<code>settled_samples</code>	<code>int</code>	3	Number of consecutive stable samples required to confirm completion
<code>joint_tolerance_deg</code>	<code>double</code>	0.2	⚠️ Deprecated, no longer effective
<code>cartesian_position_tolerance_mm</code>	<code>double</code>	1.0	⚠️ Deprecated, no longer effective
<code>cartesian_orientation_tolerance_deg</code>	<code>double</code>	1.0	⚠️ Deprecated, no longer effective

Usage:

```
// Use defaults
robot.MovJSync(Codroid::JointPoint::Degrees({0, 0, 90, 0, 90, 0}), 40, 100);

// Custom wait options
Codroid::MotionWaitOptions wait;
wait.timeout_s = 120.0;
wait.poll_interval_s = 0.02;
wait.settled_samples = 5;
wait.joint_tolerance_deg = 0.05;
wait.cartesian_position_tolerance_mm = 0.2;
robot.MovLSync(target, 50, 200, wait);
```

MoveTo Parameters

MoveToType

```
enum class MoveToType : int {
    Stop = -1,      // Stop MoveTo
    Home = 0,       // Go Home
    Safe = 1,       // Go to safe position
    Candle = 2,     // Go to candle position
    Packing = 3,    // Go to packing position
    Joint = 4,      // Joint planning to target
    Line = 5,       // Linear planning to target
    ResumePoint = 6 // Resume program execution
};
```

Name	Value	Description
<code>Stop</code>	-1	Stop MoveTo operation
<code>Home</code>	0	Home position

Name	Value	Description
Safe	1	Safe position
Candle	2	Candle (vertical) position
Packing	3	Packing (transport) position
Joint	4	Joint planned motion to target
Line	5	Linear planned motion to target
ResumePoint	6	Resume program execution

MoveToParams

```
struct MoveToParams {
    MoveToType type = MoveToType::Home;
    MoveToTarget target;
};
```

Property	Type	Description
type	MoveToType	Target type
target	MoveToTarget	Planned target point (only needed for Joint/Line)

Jog Parameters

JogMode

```
enum class JogMode {
    Joint = 1, // Joint jog
    Line = 2   // Linear jog
};
```

Name	Value	Description
Joint	1	Jog a single joint
Line	2	Linear jog in Cartesian space

JogParams

```
struct JogParams {
    JogMode mode = JogMode::Line;
    double speed = 0.0;
    int index = 1;
    CoordType coordType = CoordType::User;
    int coordId = 1;
};
```

Property	Type	Default	Description
mode	JogMode	Line	Jog mode: joint or linear
speed	double	0.0	Jog speed (-1~1 ratio)
index	int	1	Axis number or direction
coordType	CoordType	User	Coordinate frame type: user or tool
coordId	int	1	Coordinate frame ID

Data Types and Enums

Communication

CommandResult

Result of most TCP commands.

```
struct CommandResult {
    int id = 0;
    std::string ty;
    std::string error_msg;
    std::string raw_json;

    bool Ok() const noexcept;
};
```

Property	Type	Description
id	int	Request ID
ty	string	Response type / route
error_msg	string	Error message (empty means success)
raw_json	string	Complete response JSON

Method	Return Type	Description
Ok()	bool	Returns true if error_msg is empty

Real-Time Data

ClientRealtimeState

CRI real-time data snapshot, containing joint positions, TCP pose, status flags, etc.

Timestamp

Property	Type	Description
timestamp_ms	int64_t	Controller-side timestamp in milliseconds
data_valid	bool	Whether data is valid

Status Flags

Property	Type	Description
status1_raw	uint16_t	Controller raw status register 1
status2_raw	uint16_t	Controller raw status register 2
project_running	bool	Whether a program is currently running
project_stopped	bool	Whether the program has stopped
project_paused	bool	Whether the program is paused
enabling	bool	Whether the enable switch is active
not_enabled	bool	Robot is not in enabled state
manual_mode	bool	Currently in manual operation mode
dragging	bool	Robot is in drag teaching state
in_motion	bool	Robot has axes currently in motion
collision_stopped	bool	Robot stopped due to collision detection
in_safety_position	bool	Robot has reached safety position
has_alarm	bool	Controller has active alarm
simulation_mode	bool	Controller is in simulation mode
emergency_stop_pressed	bool	Emergency stop button is pressed
rescue_mode	bool	Robot is in collision rescue mode
auto_mode	bool	Controller is in auto mode
remote_mode	bool	Controller is in remote mode
realtime_control_mode	bool	Controller is in real-time control mode
cri_error_code	uint8_t	CRI protocol error code

Joint Data

Property	Type	Description
joint_position	vector<double>	Current joint angles in degrees
joint_velocity	vector<double>	Current joint angular velocities
joint_output_torque	vector<double>	Current joint output torque percentages
joint_external_force	vector<double>	External forces detected at each joint

TCP Data

Property	Type	Description
tcp_pose	vector<double>	Tool center point pose, XYZ in mm, ABC in degrees
tcp_velocity	vector<double>	TCP six-dimensional velocity
tcp_linear_velocity_mm_s	double	TCP linear velocity scalar

External Axis

Property	Type	Description
external_axis_position	vector<double>	External axis (e.g., track, turntable) position array

IO Related

ClientIoInfo / ClientRegisterInfo

```
struct ClientIoInfo {
    std::string type;
    int port = 0;
    double value = 0.0;
};

struct ClientRegisterInfo {
    int address = 0;
    double value = 0.0;
};
```

Property	Type	Description
type	string	IO type string
port	int	Port number
value	double	Value
address	int	Register address

Robot Settings

ClientRobotFrame / ClientRobotPayload

```
struct ClientRobotFrame {  
    int id = 0;  
    double x = 0.0, y = 0.0, z = 0.0;  
    double a = 0.0, b = 0.0, c = 0.0;  
};  
  
struct ClientRobotPayload {  
    int id = 0;  
    double m = 0.0;  
    double mx = 0.0, my = 0.0, mz = 0.0;  
};
```

ClientRobotFrame properties:

Property	Type	Description
id	int	Unique frame number
x, y, z	double	Position offset (mm)
a, b, c	double	Orientation angles (degrees)

ClientRobotPayload properties:

Property	Type	Description
id	int	Unique payload configuration number
m	double	Payload mass (kg)
mx, my, mz	double	Center of mass offset (mm)

ClientRobotParameters

```
struct ClientRobotParameters {  
    bool valid = false;  
    int default_tool_id = 0;  
    int default_payload_id = 0;  
    int default_coordinate_id = 0;  
    double max_payload = 0.0;  
    std::vector<ClientRobotFrame> tool;  
    std::vector<ClientRobotPayload> payload;  
    std::vector<ClientRobotFrame> coordinate;  
};
```

Property	Type	Description
<code>valid</code>	<code>bool</code>	Whether data is valid (false on failure)
<code>default_tool_id</code>	<code>int</code>	Currently active tool frame number
<code>default_payload_id</code>	<code>int</code>	Currently active payload configuration number
<code>default_coordinate_id</code>	<code>int</code>	Currently active user coordinate frame number
<code>max_payload</code>	<code>double</code>	Maximum allowed payload mass (kg)
<code>tool</code>	<code>vector<ClientRobotFrame></code>	All configured tool frames
<code>payload</code>	<code>vector<ClientRobotPayload></code>	All configured payload parameters
<code>coordinate</code>	<code>vector<ClientRobotFrame></code>	All configured user coordinate frames

Publish/Subscribe

ClientPublishNotification

```
struct ClientPublishNotification {
    std::string ty;
    std::string db_json;
    std::string raw_json;
};
```

Property	Type	Description
<code>ty</code>	<code>string</code>	Topic type
<code>db_json</code>	<code>string</code>	Business payload JSON string
<code>raw_json</code>	<code>string</code>	Complete JSON text

ClientPublishSubscription

```
class ClientPublishSubscription {
public:
    void Dispose();
    bool IsValid() const noexcept;
};
```

Method	Description
<code>Dispose()</code>	End subscription early (similar to destruction)

Method	Description
<code>IsValid()</code>	Whether still bound to valid internal resources

Global Variables

Variable

```
struct Variable {
    std::string val;
    std::string nm;

    template<typename T>
    Variable(const T& value, const std::string& note = "");
};
```

Property	Type	Description
<code>val</code>	<code>string</code>	Variable JSON value string
<code>nm</code>	<code>string</code>	Variable remark name

Kinematics

FKParams / IKParams

```
struct FKParams {
    std::vector<double> jp;
    std::vector<double> coor, tool, ep;
};

struct IKParams {
    std::vector<double> cp;
    std::vector<double> rj;
    std::vector<double> ep;
};
```

FKParams properties:

Property	Type	Description
<code>jp</code>	<code>vector<double></code>	6 joint angles (degrees)
<code>coor</code>	<code>vector<double></code>	User coordinate frame
<code>tool</code>	<code>vector<double></code>	Tool coordinate frame
<code>ep</code>	<code>vector<double></code>	External axis positions

IKParams properties:

Property	Type	Description
cp	vector<double>	TCP pose [x,y,z,rx,ry,rz] (mm + degrees)
rj	vector<double>	Reference joint angles (degrees)
ep	vector<double>	External axis positions

RelativePoseParams

```
struct RelativePoseParams {  
    std::vector<double> pos;  
    std::vector<double> offset;  
    CoordType coordType = CoordType::Tool;  
    std::vector<double> posCoord;  
    std::vector<double> coord;  
};
```

Property	Type	Description
pos	vector<double>	Current TCP pose in world coordinates
offset	vector<double>	[dx,dy,dz,drx,dry,drz] offset
coordType	CoordType	User or tool coordinate frame
posCoord	vector<double>	TCP pose in position coordinate frame
coord	vector<double>	User coordinate frame definition

Exceptions

CodroidException / CodroidCommandException

```
class CodroidException : public std::runtime_error {  
public:  
    explicit CodroidException(const std::string& message);  
};  
  
class CodroidCommandException : public CodroidException {  
public:  
    int request_id() const noexcept;  
    const std::string& command_ty() const noexcept;  
    const std::string& controller_error() const noexcept;  
    const std::string& raw_response_json() const noexcept;  
};
```

CodroidCommandException properties:

Property	Type	Description
<code>request_id()</code>	<code>int</code>	Identifier for matching request with response
<code>command_ty()</code>	<code>string</code>	String identifier indicating which CRI command failed
<code>controller_error()</code>	<code>string</code>	Controller's error description
<code>raw_response_json()</code>	<code>string</code>	Complete response JSON for further diagnosis

Firmware Version Constant

```
inline constexpr const char* MinControllerFirmware = "2.3.3.43";
```


Parameter	Type	Default	Description
<code>controller_ip</code>	<code>string</code>	—	Robot controller IP address
<code>controller_udp_port</code>	<code>int</code>	9030	UDP port for sending commands
<code>convert_to_si</code>	<code>bool</code>	<code>true</code>	If <code>true</code> , converts degrees to radians and mm to meters before sending

SendCommand

```
void SendCommand(const std::array<double, 6>& position6, TrajectorySpace space);
```

Parameter	Type	Description
<code>position6</code>	<code>array<double, 6></code>	Target position, must be exactly 6 elements
<code>space</code>	<code>TrajectorySpace</code>	Coordinate space: <code>Joint</code> or <code>Cartesian</code>

```
CriRealtimeDispatcher dispatcher("192.168.8.136");
dispatcher.SendCommand({0, 0, 90, 0, 90, 0}, TrajectorySpace::Joint);
```

SendTrajectory

```
void SendTrajectory(const std::vector<TrajectoryPoint>& trajectory, TrajectorySpace space,
int period_ms,
    const std::atomic<bool>*& cancel = nullptr);
```

Parameter	Type	Description
<code>trajectory</code>	<code>vector<TrajectoryPoint></code>	Trajectory point sequence to send
<code>space</code>	<code>TrajectorySpace</code>	Coordinate space: <code>Joint</code> or <code>Cartesian</code>
<code>period_ms</code>	<code>int</code>	Time interval between adjacent points (milliseconds)
<code>cancel</code>	<code>atomic<bool>*</code>	Cancellation flag (optional)

```
CriRealtimeDispatcher dispatcher("192.168.8.136", 9030, true);
dispatcher.SendTrajectory(trajectory, TrajectorySpace::Joint, 4);
```

Close / IsOpen

```
void Close() noexcept;
bool IsOpen() const noexcept;
```

TrajectoryGenerator

Offline trajectory generation.

```
#include "Codroid/trajectory_generator.hpp"
```

Generate

```
static std::vector<TrajectoryPoint> Generate(const std::array<double, 6>& start,  
                                             const std::array<double, 6>& target,  
                                             const TrajectoryRequest& request);
```

Parameter	Type	Description
start	array<double, 6>	Start position
target	array<double, 6>	Target position
request	TrajectoryRequest	Trajectory generation parameters

Returns: `vector<TrajectoryPoint>` — Trajectory point sequence

```
auto start = std::array<double, 6>{0, 0, 90, 0, 90, 0};
auto target = std::array<double, 6>{10, 20, 45, 0, 60, 30};

Codroid::TrajectoryRequest request;
request.space = Codroid::TrajectorySpace::Joint;
request.duration_s = 2.0;
request.frequency_hz = 250;

auto trajectory = Codroid::TrajectoryGenerator::Generate(start, target, request);
```

GenerateMultiSegment

```
static std::vector<TrajectoryPoint> GenerateMultiSegment(const  
std::vector<std::array<double, 6>>& waypoints,  
const TrajectoryRequest& request);
```

TrajectoryRequest

```
struct TrajectoryRequest {
    TrajectorySpace space = TrajectorySpace::Joint;
    TrajectoryProfile profile = TrajectoryProfile::Trapezoidal;
    double duration_s = 1.0;
    double frequency_hz = 250.0;
    double max_velocity = 0.0;
    double max_acceleration = 0.0;
    double max_jerk = 0.0;
};
```

Property	Type	Default	Description
space	TrajectorySpace	Joint	Trajectory coordinate space
profile	TrajectoryProfile	Trapezoidal	Motion curve type
duration_s	double	1.0	Total duration (seconds)
frequency_hz	double	250.0	Sampling frequency (Hz)
max_velocity	double	0.0	Maximum velocity
max_acceleration	double	0.0	Maximum acceleration
max_jerk	double	0.0	Maximum jerk

TrajectorySpace / TrajectoryProfile

```
enum class TrajectorySpace { Joint, Cartesian };
enum class TrajectoryProfile { Cubic, Trapezoidal };
```

Name	Description
TrajectorySpace::Joint	Joint space: positions are joint angles
TrajectorySpace::Cartesian	Cartesian space: positions are tool poses
TrajectoryProfile::Cubic	Cubic polynomial curve: smooth acceleration/deceleration
TrajectoryProfile::Trapezoidal	Trapezoidal velocity curve

Complete Example

```
#include "Codroid/client.hpp"
#include "Codroid/cri_realtime_dispatcher.hpp"
#include "Codroid/trajectory_generator.hpp"

// 1. Connect
```

```

Codroid::CodroidClient robot;
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");

// 2. Start CRI data push
robot.StartCriDataPush("192.168.8.150", 9030);
robot.WaitForCriData(5.0);

// 3. Start real-time control
robot.StartCriControl(1, 4, 5);

// 4. Wait for real-time control mode
while (!robot.GetRobotRealtimeState().realtime_control_mode) {
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}

// 5. Generate trajectory
auto state = robot.GetRobotRealtimeState();
std::array<double, 6> start;
for (int i = 0; i < 6; i++) start[i] = state.joint_position[i];

Codroid::TrajectoryRequest request;
request.space = Codroid::TrajectorySpace::Joint;
request.duration_s = 2.0;
request.frequency_hz = 250;

auto trajectory = Codroid::TrajectoryGenerator::Generate(
    start, {10, 0, 90, 0, 90, 0}, request);

// 6. Send trajectory
Codroid::CriRealtimeDispatcher dispatcher("192.168.8.136", 9030, true);
dispatcher.SendTrajectory(trajectory, Codroid::TrajectorySpace::Joint, 4);

// 7. Cleanup
robot.StopCriControl();
robot.StopCriDataPush();
robot.Disconnect();

```

IO and Register API Reference

Digital IO

GetDi / GetDo

```
int GetDi(int port, int id = 1);  
int GetDo(int port, int id = 1);
```

SetDo

```
CommandResult SetDo(int port, int value, int id = 1);
```

Analog IO

GetAi / GetAo

```
double GetAi(int port, int id = 1);  
double GetAo(int port, int id = 1);
```

SetAo

```
CommandResult SetAo(int port, double value, int id = 1);
```

Registers

GetRegisterValue / GetRegisterValues

```
double GetRegisterValue(int address, int id = 1);  
std::vector<ClientRegisterInfo> GetRegisterValues(const std::vector<int>& addresses, int id  
= 1);
```

SetRegisterValue

```
CommandResult SetRegisterValue(int address, double value, int id = 1);
```

Complete Example

```
#include "Codroid/client.hpp"  
  
Codroid::CodroidClient robot;  
robot.ConnectRemoteAndSwitchOn("192.168.8.136", 9001, "192.168.8.150");
```

```
// IO operations
int di0 = robot.GetDi(0);
robot.SetDo(10, di0);

double ai0 = robot.GetAi(0);
robot.SetAo(0, 3.3);

// Register operations
double value = robot.GetRegisterValue(49100);
robot.SetRegisterValue(49100, value + 1);

// Batch read
std::vector<int> addresses = {49100, 49101, 49102};
auto values = robot.GetRegisterValues(addresses);

robot.Disconnect();
```


Utilities API Reference

Publish/Subscribe

SubscribePublishTopic

```
ClientPublishSubscription SubscribePublishTopic(  
    std::string topicTy,  
    std::function<void(const ClientPublishNotification&)> handler,  
    int tc_milliseconds = 100);
```

Available Topics

Topic	Description
publish/ProjectState	Project running state
publish/VarUpdate	Variable update
publish/RobotStatus	Robot status
publish/RobotPosture	Robot posture
publish/RobotCoordinate	Coordinate system
publish/Log	Log
publish/Error	Error

PublishTopics Constants

```
struct PublishTopics {  
    static constexpr const char* ProjectState = "publish/ProjectState";  
    static constexpr const char* VarUpdate = "publish/VarUpdate";  
    static constexpr const char* RobotStatus = "publish/RobotStatus";  
    static constexpr const char* RobotPosture = "publish/RobotPosture";  
    static constexpr const char* RobotCoordinate = "publish/RobotCoordinate";  
    static constexpr const char* Log = "publish/Log";  
    static constexpr const char* Error = "publish/Error";  
};
```

Global Variables

GetGlobalVars / SaveGlobalVars / RemoveGlobalVars

```
nlohmann::json GetGlobalVars(int id = 1);
CommandResult SaveGlobalVars(const std::map<std::string, Variable>& vars, int id = 1);
CommandResult RemoveGlobalVars(const std::vector<std::string>& names, int id = 1);
```

Kinematics

ForwardKinematics / InverseKinematics

```
std::vector<double> ForwardKinematics(const FKParams& params, int id = 1);
std::vector<double> InverseKinematics(const IKParams& params, int id = 1);
```

CalculateRelativePose

```
std::vector<double> CalculateRelativePose(const RelativePoseParams& params, int id = 1);
```

CposToCpos / CposToCposPose (v2.1.8+)

Coordinate system transformation: convert a TCP pose from coordinate system 1 + tool 1 to coordinate system 2 + tool 2. Protocol `Robot/cpostocpos`.

```
// Returns raw double array
std::vector<double> CposToCpos(const std::vector<double>& cp,
                               const std::vector<double>& coor1, const std::vector<double>&
tool1,
                               const std::vector<double>& coor2, const std::vector<double>&
tool2,
                               int id = 1);

// Returns CartesianPoint
CartesianPoint CposToCposPose(const CartesianPoint& cp,
                              const std::vector<double>& coor1, const std::vector<double>&
tool1,
                              const std::vector<double>& coor2, const std::vector<double>&
tool2,
                              int id = 1);
```

Parameter	Description
<code>cp</code>	Current TCP pose <code>[x, y, z, a, b, c]</code> (mm+deg)
<code>coor1</code>	Source coordinate system <code>[x, y, z, a, b, c]</code>
<code>tool1</code>	Source tool <code>[x, y, z, a, b, c]</code>

Parameter	Description
<code>coord2</code>	Target coordinate system <code>[x, y, z, a, b, c]</code>
<code>tool2</code>	Target tool <code>[x, y, z, a, b, c]</code>

```
auto result = robot.CposToCpos({400, 200, 500, 180, 0, 90},
                               {0, 0, 0, 0, 0, 0}, {0, 0, 0, 0, 0, 0},
                               {100, 0, 0, 0, 0, 0}, {0, 0, 100, 0, 0, 0});
```

RS485 Communication (v2.1.8+)

Rs485Init

```
CommandResult Rs485Init(int baudrate, RS485StopBits stopBit = RS485StopBits::One,
                        RS485Parity parity = RS485Parity::None, int id = 1);
```

Initialize end-effector RS485 communication.

Rs485Flush

```
CommandResult Rs485Flush(int id = 1);
```

Flush RS485 read buffer.

Rs485Read

```
nlohmann::json Rs485Read(int length, int timeout = 5000, int id = 1);
```

Read RS485 data. `length` max 128 bytes, `timeout` max 3000ms.

Rs485Write

```
CommandResult Rs485Write(const std::vector<uint8_t>& data, int id = 1);
```

Write RS485 data. `data` max 127 bytes.

```
robot.Rs485Init(115200, Codroid::RS485StopBits::One, Codroid::RS485Parity::None);
robot.Rs485Flush();
auto data = robot.Rs485Read(7, 1000);
robot.Rs485Write({0x01, 0x03, 0x00, 0x00, 0x00, 0x01, 0x84, 0x0A});
```

Project Control Extensions (v2.1.8+)

SetStartLine / ClearStartLine

```
CommandResult SetStartLine(int line, int id = 1);  
CommandResult ClearStartLine(int id = 1);
```

Set/clear project execution start line.

GetProjectVar

```
nlohmann::json GetProjectVar(int id = 1);
```

Get current project variable values (only valid during project execution).

GetGlobalVarsCatalog

```
nlohmann::json GetGlobalVarsCatalog(int id = 1);
```

Get global variables catalog (same protocol as `GetGlobalVars`, parsed into `name → {Value, Remark}`).

Register Extensions (v2.1.8+)

SetExtendArrayType / RemoveExtendArray

```
CommandResult SetExtendArrayType(int index, ExtendArrayType type, int id = 1);  
CommandResult RemoveExtendArray(int index, int id = 1);
```

Set/reset extended array element type.

Robot Settings Extensions (v2.1.8+)

SaveUserCoordinateFrames

```
CommandResult SaveUserCoordinateFrames(const std::vector<ClientRobotFrame>& frames, int id = 1);
```

Directly save the complete user coordinate frame table (19.6), aligned with `SaveToolFrames` / `SavePayloadFrames`.

Console UTF-8

ConsoleUtf8

```
#include "Codroid/console_utf8.hpp"

// Windows console UTF-8 support
#ifdef _WIN32
Codroid::ConsoleUtf8::InitConsoleUtf8();
#endif
```

Complete Example

```
#include "Codroid/client.hpp"

// Publish/Subscribe
auto sub = robot.SubscribePublishTopic(
    Codroid::PublishTopics::RobotStatus,
    [](const Codroid::ClientPublishNotification& n) {
        std::cout << "Status: " << n.db_json << std::endl;
    }
);

// Global variables
robot.SaveGlobalVars({
    {"counter", Codroid::Variable(42, "Counter")},
    {"name", Codroid::Variable(std::string("test"), "Name")}
});

// Kinematics
Codroid::FKParams fk({0, 0, 90, 0, 90, 0});
auto tcp = robot.ForwardKinematics(fk);

Codroid::IKParams ik({400, 0, 300, 180, 0, 0});
ik.rj = {0, 0, 90, 0, 90, 0};
auto joints = robot.InverseKinematics(ik);
```